

Statement 2

3) FUNCTION ARCHITECTURE AND COMPONENT:

This section highlights the functional modules—or the specific building blocks—that make up the ATM simulator. It explains how each feature operates under the hood to handle data and user requests.

3.1) Secure Multi-Attempt User Authentication Matrix:

This component acts as the security gatekeeper. Before revealing any financial options, the system prompts the user for a secret PIN. It uses a loop coupled with a counter variable to track incorrect entries. If the user fails to provide the correct PIN within a set number of tries (usually 3), the system locks out the user to prevent unauthorized access.

3.2)Balance Retrieval and Display Format Architecture :

This module handles reading and displaying the user's financial state. It takes the current numerical value stored in the `account_balance` variable and outputs it in a clean, human-readable format. This involves applying string formatting to display the currency symbol (\$ or ₹) alongside appropriate commas and decimal places.

3.3)Deposit Logic and Value Filtering:

This function handles incoming funds. Before updating the balance, the logic filters the input to ensure it is a valid, positive number. If a user attempts to input a negative number or zero, the system catches the anomaly, rejects the transaction, and prompts the user to enter a legitimate amount.

3.4) Withdrawal Safeguards (Overdraft and Negative Input Checking):

This is a critical defensive module designed to protect the account from draining into a negative state. When a user requests cash, the system runs a two-step validation: first, it checks that the requested amount is a positive number; second, it verifies that the request does not exceed the current account balance. If the amount is too high, the transaction is immediately blocked.

3.5)Graceful Session Exits and Program State Resetting:

This component manages the proper closing of the user session. When the user selects the exit option, the system clears any temporary session flags, prints a friendly closing thank-you message, and safely shuts down the command-line interface. This ensures the program terminates smoothly without leaving behind dangling code loops.

4) SOURCE CODE IMPLEMENTATION :

This section bridges the theoretical concepts and the actual execution of the project by highlighting how the code is structured and written. It explains the core syntax and variable setup required to make the simulator functional.

4.1)Variable Initialization & State Setup:

At the very beginning of the script, the application defines the starting environment. It initializes variables to store the user's secret data (like `user_pin`), the maximum number of login tries allowed (`max_attempts`), and a floating-point number to track the financial state (`account_balance`).

4.2)The Guardrail Code Block (Authentication Loop):

The first active code block handles login security. It uses a while loop that counts down with each failed login attempt. Inside this loop, an if-else statement compares user input to the correct PIN. If a match is found, a break statement ends the login screen; if the countdown hits zero, an `exit()` function shuts down the entire script.

4.3)The Main Application Engine (Menu Loop):

Once logged in, the program moves into a permanent while True loop. This acts as the motor of the application, keeping the screen alive so the user can perform multiple actions back-to-back. Every time a

transaction finishes, the loop cycles back to print the main menu choices (Balance, Deposit, Withdraw, Exit).

4.4) Logical Routing Matrix (if-elif-else):

Inside the menu loop, the code evaluates the user's numeric choice (1 through 4). This routing is handled by a clean chain of if (choice 1), elif (choices 2 and 3), and an else catch-all at the very bottom. The else block serves as an error-handler, catching typos or invalid numbers and prompting the user to try choosing a menu option again without crashing the script.